DYNAMIC PROGRAMMING

By Anand Kumar Rohal

Contents

1 1 – D P 3

1.1 Fibonacci series 3

1.2 Climb stairs 4

1.3 Frog jump 5

1.4 Frog jump with k distance 6

1.5 Maximum sum without adjacent elements 7

1.6 House robber 8

2 2 – D DP General 9

2.1 Ninjas training 9

3 DP on Grids 11

3.1 Grid unique path 11

3.2 Unique path with obstacles 12

3.3 Minimum path sum grid 14

3.4 Triangle grid 16

3.5 Minimum/Maximum falling path sum 18

3.6 Cherry pick II 20

4 DP on subsequences 22

4.1 Subset sum equal to target 22

4.2 Partition array into equal sum 24

4.3 Partition array in subsets with minimum difference 26

4.4 Number of subsets equal to target 28

4.5 0/1 knapsack standard 30

4.6 Minimum coins 32

4.7 Target sum 33

4.8 Coin change II 34

4.9 Unbounded knapsack 35

4.10 Rod cutting 37

5 DP on strings 38

5.1 LCS 38

6 DP on stocks 40

6.1 Best time to buy and sell stocks 42

6.2 Buy and sell stocks II 43

6.3 Buy sell stocks III 44
6.4 Buy and stocks IV 45
6.5 Buy sell stocks with cooldown 46
6.6 Buy and sell stocks with transaction fell 47

1 1 – D P

1.1 Fibonacci series

Question – Given a positive integer n, find the nth fibonacci number. Since the answer can be very large, return the answer modulo 1000000007.

Code –

Recursive –

```
li f(li n){
    if (n <= 1) return n;
    return (f(n-1)%m+ f(n-2)%m)%m;
}
```

Memoization –

```
li f(li n,vi &dp){
    if (n <= 1) return n;
    if (dp[n] != -1) return dp[n];
    return (f(n-1,dp)%m + f(n-2,dp)%m)%m;
}
```

Tabulation –

```
li f(li n){
    vi dp(n+1);
    dp[0] = 0;
    dp[1] = 1;
    for (int i=2;i<n+1;i++)
        dp[i] = (dp[i-1] + dp[i-2])%m;
    return dp[n];
}
```

Space optimization –

```
li f(li n){
    li p = 0;
    li c = 1;
    li nth = 1;
    for (int i = 2;i<n+1;i++){
        nth = (p + c)%m;
        p = c;
        c = nth;
    }
    return nth;
}
```

1.2 Climb stairs

Question – You have been given a number of stairs. Initially, you are at the 0th stair, and you need to reach the Nth stair. Each time you can either climb one step or two steps. You are supposed to return the number of distinct ways in which you can climb from the 0th step to Nth step.

Code –

Recursive –

```
int f(int n){
    if (n <= 1) return n;
    return (f(n-1) + f(n-2))%m;
}
```

Memoization –

```
int f(int n,vi &dp){
    if (n <= 1) return n;
    if (dp[n] != -1) return dp[n];
    return dp[n] = (f(n-1,dp) + f(n-2,dp))%m;
}
```

Tabulation –

```
int f(int n){
    vi dp(n+1);
    dp[0] = 0;
    dp[1] = 1;
    for (int i=2;i < n+1;i++)
        dp[i] = (dp[i-1] + dp[i-2])%m;
    return dp[n];
}
```

Space optimization –

```
int f(int n){
    int p = 0,c = 1;
    int a = p+c;
    for (int i = 2;i <= n;i++){
        a = (p+c)%m;
        p = c;
        c = a;
    }
    return a;
}
```

1.3 Frog jump

1.4 Frog jump with k distance

1.5 Maximum sum without adjacent elements

1.6 House robber

2 2 – D DP General

2.1 Ninjas training

Question – Ninja is planning this 'N' days-long training schedule. Each day, he can perform any one of these three activities. (Running, Fighting Practice or Learning New Moves). Each activity has some merit points on each day. As Ninja has to improve all his skills, he can't do the same activity in two consecutive days. Can you help Ninja find out the maximum merit points Ninja can earn?

Code –

Recursive –

```
// starting call send l = -1
int f(int n,int l,vvi &points){
    if (!n) return 0;
    int maxi = 0;
    for (int i = 0; i < 3;i++){
        if (l != i){
            int activity =
            points[n-1][i] + f(n-1,i,points);
            maxi = max(maxi,activity);
        }
    }
    return maxi;
}
```

Memoization –

```
// dp arr take n*4 and pass l = 3 in first call
int f(int n,int l,vvi &points,vvi &dp){
    if (!n) return 0;
    if (dp[n][l] != -1) return dp[n][l];
    int maxi = 0;
    for (int i = 0;i < 3;i++){
        if (l != i){
            int activity = points[n-1][i] + f(n-1,i,points,dp);
            maxi = max(maxi,activity);
        }
    }
    return dp[n][l] = maxi;
}
```

Tabulation –

```
int f(int n,vvi &points){
    vvi dp(n,vi(3,-1));
    for (int i = 0; i < 3;i++)
        dp[0][i] = points[0][i];
```

```
    for (int i = 1; i < n;i++){
        for (int j = 0; j < 3;j++){
            int lz = j == 0? 0:dp[i-1][0];
            int lo = j == 1? 0:dp[i-1][1];
            int lt = j == 2? 0:dp[i-1][2];
            dp[i][j] = points[i][j] + max(lz,max(lo,lt));
        }
    }
    return *max_element(dp[n-1].begin(),dp[n-1].end());
}
```

Space optimization –

```
int f(int n,vvi &points){
    vi p(3), c(3);
    for (int i = 0;i < 3;i++)
        p[i] = points[0][i];

    for (int i = 1; i < n;i++){
        for (int j = 0; j < 3;j++){
            int lz = j == 0? 0:p[0];
            int lo = j == 1? 0:p[1];
            int lt = j == 2? 0:p[2];
            c[j] = points[i][j] + max(lz,max(lo,lt));
        }
        p = c;
    }
    return *max_element(c.begin(),c.end());
}
```

3 DP on Grids

3.1 Grid unique path

Question – Given a A X B matrix with your initial position at the top-left cell, find the number of possible unique paths to reach the bottom-right cell of the matrix from the initial position.

Code –

Recursive –

```
int f(int a,int b){
    if (!a && !b) return 1;
    if (a < 0 or b < 0) return 0;
    return f(a-1,b) + f(a,b-1);
}
```

Memoization –

```
int f(int a,int b,vvi &dp){
    if (!a && !b) return 1;
    if (a < 0 || b < 0) return 0;
    if (dp[a][b] != -1) return dp[a][b];
    return dp[a][b] = f(a-1,b,dp) + f(a,b-1,dp);
}
```

Tabulation –

```
int f(int a,int b){
    vvi dp(a,vi(b));
    for(int j = 0; j < b; j++) dp[0][j] = 1;
    for (int i = 0; i < a; i++) dp[i][0] = 1;

    for (int i = 1; i < a; i++){
        for (int j = 1; j < b;j++){
            dp[i][j] = dp[i-1][j] + dp[i][j-1];
        }
    }
    return dp[a-1][b-1];
}
```

Space optimization –

```
int f(int a,int b){
    vi p(b,1), c(b,1);
    for (int i = 1; i < a; i++){
        for (int j = 1;j < b;j++){
            c[j] = p[j] + c[j-1];
        }
        p = c;
    }
    return c[b-1]; }
```

3.2 Unique path with obstacles

Question – You are given an m x n integer array grid. There is a robot initially located at the top-left corner (i.e., grid[0][0]). The robot tries to move to the bottom-right corner (i.e., grid[m - 1][n - 1]). The robot can only move either down or right at any point in time.

An obstacle and space are marked as 1 or 0 respectively in grid. A path that the robot takes cannot include any square that is an obstacle.

Return the number of possible unique paths that the robot can take to reach the bottom-right corner.

Code –

Recursive –

```
int f(int x,int y,vvi &grid){
    if (x < 0 or y < 0)  return 0;
    if (grid[x][y] == 1) return 0;
    if (x == 0 and y == 0) return 1;
    return f(x-1,y,grid) + f(x,y-1,grid);
}
```

Memoization –

```
int f(int x,int y,vvi &grid,vvi &dp){
    if (x < 0 or y < 0)  return 0;
    if (grid[x][y] == 1) return 0;
    if (x == 0 and y == 0) return 1;
    if (dp[x][y] != -1) return dp[x][y];
    return dp[x][y] = f(x-1,y,grid,dp) + f(x,y-1,grid,dp);
}
```

Tabulation –

```
int f(vvi &grid){
    int xLen = grid.size(), yLen = grid[0].size();
    vvi dp(xLen,vi(yLen,-1));
    for (int i = 0; i < xLen;i++){
        for (int j = 0; j < yLen;j++){
            if (i == 0 and j == 0){
                dp[0][0] = grid[0][0] ? 0:1;
            }
            else {
                int left = 0, down = 0;
                if (j > 0)
                    left = grid[i][j]?0:dp[i][j-1];
                if (i > 0)
                    down = grid[i][j]?0:dp[i-1][j];
                dp[i][j] = left + down;
            }
        }
```

```
    }
    return dp[xLen-1][yLen-1];
}
```

Space optimization –

```
int f(vvi &grid){
    int xLen = grid.size(), yLen = grid[0].size();
    vi p(yLen),c(yLen);
    for (int i = 0; i < xLen;i++){
        for (int j = 0; j < yLen;j++){
            if  (i==0 and j==0) {
                c[0] = !grid[0][0];
                continue;
            }
            int left = 0, down = 0;
            if (j > 0) left = (!grid[i][j])*(c[j-1]);
            if (i > 0) down = (!grid[i][j])*(p[j]);
            c[j] = left + down;
        }
        p = c;
    }
    return c[yLen-1];
}
```

3.3 Minimum path sum grid

Question – Given a cost matrix costMat[][] and a destination coordinate (i,j), we have to come up with a solution that returns the minimum sum path from (0,0) to (i,j). The condition of traversal is that we can travel only towards the right direction, below and diagonally below. The minimum sum path is the sum of all the costs of the path, including the source and destination coordinates.

Code –

Recursive –

```
int f(int x,int y,vvi &grid){
    if (!x && !y) return grid[0][0];
    if (x < 0 || y < 0) return 1e9;
    return grid[x][y] + min(f(x-1,y,grid),f(x,y-1,grid));
}
```

Memoization –

```
int f(int x,int y,vvi &grid,vvi &dp){
    if (!x && !y) return grid[0][0];
    if (x < 0 || y < 0) return 1e9;
    if (dp[x][y] != -1) return dp[x][y];
    return dp[x][y] = grid[x][y]
    + min(f(x-1,y,grid,dp),f(x,y-1,grid,dp));
}
```

Tabulation –

```
int f(vvi &grid){
    int xLen = grid.size();
    int yLen = grid[0].size();
    vvi dp(xLen,vi(yLen));
    for (int i = 0; i < xLen;i++){
        for (int j = 0; j < yLen;j++){
            if (i == 0 && j == 0) {
                dp[0][0] = grid[0][0];
                continue;
            }
            int left = j > 0? dp[i][j-1]:1e9;
            int down = i > 0? dp[i-1][j]:1e9;
            dp[i][j] = grid[i][j] + min(left,down);
        }
    }
    return dp[xLen-1][yLen-1];
}
```

Space optimization –

```
int f(vvi &grid){
    int xLen = grid.size();
```

```
    int yLen = grid[0].size();
    vi p(yLen),c(yLen);
    for (int i = 0; i < xLen; i++){
        for (int j = 0; j < yLen;j++){
            if (!i && !j){
                c[0] = grid[0][0];
                continue;
            }
            int left = j > 0?c[j-1]:1e9;
            int down = i > 0?p[j]:1e9;
            c[j] = grid[i][j] + min(left,down);
        }
        p = c;
    }
    return c[yLen-1];
}
```

3.4 Triangle grid

Question – You are given a triangular array/list 'TRIANGLE'. Your task is to return the minimum path sum to reach from the top to the bottom row.
The triangle array will have N rows and the i-th row, where 0 <= i < N will have i + 1 elements.
You can move only to the adjacent number of row below each step. For example, if you are at index j in row i, then you can move to i or i + 1 index in row j + 1 in each step.
Code –

Recursive –

```
int f(int x ,int y ,vvi &tri){
    if (x == tri.size()) return 0;
    return tri[x][y] + min(f(x+1,y,tri),f(x+1,y+1,tri));
}
```

Memoization –

```
int f(int x,int y,vvi &tri,vvi &dp){
    if (x == tri.size()) return 0;
    if (dp[x][y] != -1) return dp[x][y];
    return dp[x][y] = tri[x][y]
    + min(f(x+1,y,tri,dp),f(x+1,y+1,tri,dp));
}
```

Tabulation –

```
int f(vvi &triangle){
    int n = triangle.size();
    vvi dp(n,vi(n));
    for (int j = 0; j < n;j++)
    dp[n-1][j] = triangle[n-1][j];

    for (int i = n-2; i >= 0;i--){
        for (int j = 0; j <= i;j++){
            dp[i][j] = triangle[i][j]
            + min(dp[i+1][j],dp[i+1][j+1]);
        }
    }
    return dp[0][0];
}
```

Space optimization –

```
int f(vvi &triangle){
    int n = triangle.size();
    vi p(n),c(n);
    for (int j = 0; j < n;j++)
    p[j] = triangle[n-1][j];
    for (int i = n-2; i >= 0;i--){
        for (int j = 0; j <= i;j++){
```

```
            c[j] = triangle[i][j] + min(p[j],p[j+1]);
        }
        p = c;
    }
    return p[0];
}
```

3.5 Minimum/Maximum falling path sum

Question – You have been given an N*M matrix filled with integer numbers, find the maximum sum that can be obtained from a path starting from any cell in the first row to any cell in the last row.
From a cell in a row, you can move to another cell directly below that row, or diagonally below left or right. So from a particular cell (row, col), we can move in three directions i.e.
Down: (row+1,col)
Down left diagonal: (row+1,col-1)
Down right diagonal: (row+1, col+1)
Code –

Recursive –

```
// call f for whole last row
int f(int x,int y,vvi &matrix){
    if ( y < 0 or y >= matrix[0].size()) return -1e9;
    if (x < 0) return 0;
    int maxi = -1e9;
    for (int i = -1; i < 2; i++)
        maxi = max(maxi,matrix[x][y] + f(x-1,y-i,matrix));
    return maxi;
}
```

Memoization –

```
// ans is max of last row
int f(int x,int y,vvi &matrix,vvi &dp){
    if ( y < 0 or y >= matrix[0].size()) return -1e9;
    if (x < 0) return 0;
    if (dp[x][y] != -1) return dp[x][y];
    int maxi = -1e9;
    for (int i = -1; i < 2; i++)
        maxi = max(maxi,matrix[x][y] + f(x-1,y-i,matrix,dp));
    return dp[x][y] = maxi;
}
```

Tabulation –

```
int f(vvi &matrix){
    int xLen = matrix.size(), yLen = matrix[0].size();
    vvi dp(xLen,vi(yLen));
    for (int j = 0; j < yLen;j++)
        dp[0][j] = matrix[0][j];

    for (int i = 1;i < xLen;i++){
        for (int j = 0;j < yLen;j++){
            int down = dp[i-1][j];
            int left = j > 0?dp[i-1][j-1] :-1e9;
            int right = j < yLen -1?dp[i-1][j+1]:-1e9;
            dp[i][j] = matrix[i][j] + max(down,max(left,right));
```

```
        }
    }
    return *max_element(dp[xLen-1].begin(),dp[xLen-1].end());
}
```

Space optimization –

```
int f(vvi &matrix){
    int xLen = matrix.size(), yLen = matrix[0].size();
    vi p(yLen), c(yLen);
    for (int j = 0;j < yLen;j++)
        p[j] = matrix[0][j];

    for (int i = 1;i < xLen;i++){
        for (int j = 0;j < yLen;j++){
            int down = p[j];
            int left = j > 0?p[j-1] :-1e9;
            int right = j < yLen -1?p[j+1]:-1e9;
            c[j] = matrix[i][j] + max(down,max(left,right));

        }
        p = c;
    }
    return *max_element(p.begin(),p.end());
}
```

3.6 Cherry pick II

Question – Ninja has a 'GRID' of size 'R' X 'C'. Each cell of the grid contains some chocolates. Ninja has two friends Alice and Bob, and he wants to collect as many chocolates as possible with the help of his friends.
Initially, Alice is in the top-left position i.e. (0, 0), and Bob is in the top-right place i.e. (0, 'C' - 1) in the grid. Each of them can move from their current cell to the cells just below them. When anyone passes from any cell, he will pick all chocolates in it, and then the number of chocolates in that cell will become zero. If both stay in the same cell, only one of them will pick the chocolates in it.
If Alice or Bob is at (i, j) then they can move to (i + 1, j), (i + 1, j - 1) or (i + 1, j + 1). They will always stay inside the 'GRID'.
Your task is to find the maximum number of chocolates Ninja can collect with the help of his friends by following the above rules.
Code –

Recursive –

```
int f(int a,int b,int r,vv &grid){
    if (r == grid.size()) return 0;
    if (a==-1 || a==grid[0].size() || b==-
1 || b==grid[0].size()) return -1e9;
    int maxi = -1e9;
    for (int del_a = -1;del_a <= 1;del_a++){
        for (int del_b = -1;del_b <= 1;del_b++){
            if (a == b) maxi = max(maxi,
            grid[r][b]+f(a+del_a,b+del_b,r+1,grid));
            else maxi = max(maxi,grid[r][a]+
            grid[r][b]+f(a+del_a,b+del_b,r+1,grid));
        }
    }
    return maxi;
}
```

Memoization –

```
int f(int a,int b,int r,vv &grid,vvv &dp){
    if (r == grid.size()) return 0;
    if (a==-1 || a==grid[0].size() ||
    b==-1 || b==grid[0].size()) return -1e9;
    if (dp[a][b][r] != -1) return dp[a][b][r];
    int maxi = -1e9;
    for (int del_a = -1;del_a <= 1;del_a++){
        for (int del_b = -1;del_b <= 1;del_b++){
            if (a == b) maxi =
            max(maxi,grid[r][b]+f(a+del_a,b+del_b,r+1,grid,dp));
            else maxi = max(maxi,grid[r][a]+grid[r][b]+
            f(a+del_a,b+del_b,r+1,grid,dp));
        }
```

```
    }
    return dp[a][b][r] = maxi;
}
```

Tabulation –

Space optimization –

4 DP on subsequences

4.1 Subset sum equal to target

Question – You are given an array/list 'ARR' of 'N' positive integers and an integer 'K'. Your task is to check if there exists a subset in 'ARR' with a sum equal to 'K'.

Note: Return true if there exists a subset with sum equal to 'K'. Otherwise, return false.

Code –

Recursive –

```
bool f(int n, int k,vector<int> &arr){
    if (k == 0) return 1;
    if (n == 0) return 0;
    int curr_element = arr[n-1];
    int take = 0;
    if (curr_element <= k) take = f(n-1,k-curr_element,arr);
    int not_take = f(n-1,k,arr);
    return take or not_take;
}
```

Memoization –

```
bool f(int n,int k,vi &arr,vvi &dp){
    if (k == 0) return 1;
    if (n == 0 or k < 0) return 0;
    if (dp[n][k] != -1) return dp[n][k];
    int curr_element = arr[n-1];
    int take = f(n-1,k-curr_element,arr,dp);
    int not_take = f(n-1,k,arr,dp);
    return dp[n][k] = take or not_take;
}
```

Tabulation –

```
bool f(int n,int k,vi &arr){
    vvi dp(n+1,vi(k+1,-1));
    for  (int i = 0; i < dp.size();i++) dp[i][0] = 1;
    for  (int j = 1; j < dp[0].size();j++) dp[0][j] = 0;
    for (int i = 1; i < n+1;i++){
        for (int j = 1; j < k + 1;j++){
            int not_take = dp[i-1][j];
            int curr_ele = arr[i-1];
            int take = curr_ele <= j ? dp[i-1][j-curr_ele]:0;
            dp[i][j] = take or not_take;
        }
    }
    return dp[n][k];
}
```

Space optimization –

```
bool f(int n,int k,vi &arr){
    vi c(k+1),p(k+1,0);
    p[0] = c[0] = 1;
    for (int i = 1; i < n+1;i++){
        c[0] = 1;
        for (int j = 1; j < k+1;j++){
            int curr_element = arr[i-1];
            int take = curr_element <= j? p[j-curr_element]:0;
            int not_take = p[j];
            c[j] = take or not_take;
        }
        p = c;
    }
    return c[k];
}
```

4.2 Partition array into equal sum

Question – Given an array arr[] of size N, check if it can be partitioned into two parts such that the sum of elements in both parts is the same. If yes return 1 else 0.

Code –

Recursive –

```
// k = sum/2 only when sum is even
int f(int n,int k,int arr[]){
    if (!k) return 1;
    if (!n or k < 0) return 0;
    int curr_element = arr[n-1];
    int take = curr_element <= k ?
    f(n-1,k - curr_element,arr) : 0;
    int not_take = f(n-1,k,arr);
    return take or not_take;
}
```

Memoization –

```
// k = sum/2 only when sum is even
int f(int n,int k,int arr[],vvi &dp){
    if (!k) return 1;
    if (!n or k < 0) return 0;
    if (dp[n][k] != -1) return dp[n][k];
    int curr_ele = arr[n-1];
    int take = curr_ele <= k ?
    f(n-1,k-curr_ele,arr,dp): 0;
    int not_take = f(n-1,k,arr,dp);
    return dp[n][k] = take or not_take;
}
```

Tabulation –

```
bool f(int n,int arr[]){
    int sum = accumulate(arr,arr+n,0);
    if (sum%2) return false;
    sum /= 2;
    vvi dp(n+1,vi(sum+1));
    for (int i = 0; i < n + 1; i++) dp[i][0] = 1;
    for (int j = 1; j < sum + 1; j++) dp[0][j] = 0;

    for (int i = 1; i < n+1;i++){
        for (int j = 1;j < sum+1;j++){
            int curr_element = arr[i-1];
            int take = curr_element <= j? dp[i-1][j-curr_element]:0;
            int not_take = dp[i-1][j];
            dp[i][j] = take or not_take;
        }
```

```
    }
    return dp[n][sum];
}
```

Space optimization –

```
bool f(int n,int arr[]){
    int sum = accumulate(arr,arr+n,0);
    if (sum%2) return false;
    sum /= 2;
    vi p(sum+1,0),c(sum+1);
    for (int i = 1; i < n+1; i++){
        p[0] = c[0] = 1;
        for (int j = 1; j < sum+1; j++){
            int curr_element = arr[i-1];
            int take = curr_element <= j?
            p[j-curr_element]:0;
            int not_take = p[j];
            c[j] = take or not_take;
        }
        p = c;
    }
    return c[sum];
}
```

4.3 Partition array in subsets with minimum difference

Question – We are given an array 'ARR' with N positive integers. We need to partition the array into two subsets such that the absolute difference of the sum of elements of the subsets is minimum.
We need to return only the minimum absolute difference of the sum of elements of the two partitions.
Code –

Recursive –

```
bool f(int n,int s, vi &arr){
    if (s == 0) return 1;
    if (n == 0) return 0;
    int take = arr[n-1] <= s? f(n-1,s-arr[n-1],arr): 0;
    int not_take = f(n-1,s,arr);
    return take or not_take;
}
int fSolve(vi &arr){
    int n = arr.size();
    int sum = accumulate(arr.begin(),arr.end(),0);
    vi possibleSumArr(sum+1);
    for (int i = 0; i < sum + 1;i++){
        possibleSumArr[i] = f(n,i,arr);
    }
    int minDiff = INT32_MAX;
    for (int i = 0; i < sum + 1;i++){
        if (possibleSumArr[i]){
            int currDiff = abs(sum-2*i);
            minDiff = min(minDiff,currDiff);
        }
    }
    return minDiff;
}
```

Memoization –

```
// run for 0 to sum(arr) as done above
bool f(int n,int s,vi &ar,vvi &dp){
    if (s == 0) return 1;
    if (n == 0) return 0;
    if (dp[n][s] != -1) return dp[n][s];
    int take = ar[n-1] <= s?f(n-1,s-ar[n-1],ar,dp):0;
    int not_take = f(n-1,s,ar,dp);
    return dp[n][s] = take or not_take;
}
```

Tabulation –

```
int f(vi &arr){
    int n = arr.size();
```

```
    int sum = accumulate(arr.begin(),arr.end(),0);
    vvb dp(n+1,vb(sum+1));
    for (int j = 1;j < sum+1;j++) dp[0][j] = 0;
    for (int i = 0;i < n+1;i++) dp[i][0] = 1;
    for (int i = 1; i < n+1;i++){
        for (int j = 1; j < sum+1;j++){
            bool take = arr[i-1] <= j?
            dp[i-1][j-arr[i-1]]:0;
            bool not_take = dp[i-1][j];
            dp[i][j] = take or not_take;
        }
    }
    int minDiff = INT32_MAX;
    for (int j = 0; j < sum +1;j++){
        if (dp[n][j])
        minDiff = min(minDiff,abs(sum-2*j));
    }
    return minDiff;
}
```

Space optimization –

```
int f(vi &arr){
    int n = arr.size();
    int sum = accumulate(arr.begin(),arr.end(),0);
    vb p(sum+1,0),c(sum+1);
    for (int i=1;i<n+1;i++){
        p[0] = c[0] = 1;
        for (int j=1;j<sum+1;j++){
            int take = arr[i-1]<=j?p[j-arr[i-1]]:0;
            int not_take = p[j];
            c[j] = take or not_take;
        }
        p = c;
    }
    int minDiff = INT32_MAX;
    for (int i=0;i<sum+1;i++){
        if(p[i]) minDiff = min(minDiff,abs(sum-2*i));
    }
    return minDiff;
}
```

4.4 Number of subsets equal to target

Question – Given an array arr[] of non-negative integers and an integer sum, the task is to count all subsets of the given array with a sum equal to a given sum.
Note: Answer can be very large, so, output answer modulo 109+7
Code –

Recursive –

```
int f(int n,int s,int arr[],int cz[]){
    if (s == 0) return !n?1:pow(2,cz[n-1]);
    if (n == 0 or s < 0) return 0;
    int take = f(n-1,s-arr[n-1],arr,cz);
    int not_take = f(n-1,s,arr,cz);
    return (take+not_take)%m;
}
```

Memoization –

```
int f(int n,int s,int arr[],int cz[],vvi &dp){
    if (s == 0) return !n?1:pow(2,cz[n-1]);
    if (n == 0 or s < 0) return  0;
    if (dp[n][s] != -1) return dp[n][s];
    int take = f(n-1,s-arr[n-1],arr,cz,dp);
    int not_take = f(n-1,s,arr,cz,dp);
    return dp[n][s] = (take+not_take)%m;
}
```

Tabulation –

```
int f(int arr[],int n,int sum){
    int dp[n+1][sum+1];
    int cz = 0;
    for (int i = 1; i < n+1;i++){
        if (arr[i-1] == 0) cz++;
        dp[i][0] = pow(2,cz);
    }
    for (int j = 1;j< sum+1;j++) dp[0][j] = 0;
    dp[0][0] = 1;
    for (int i = 1;i < n+1;i++){
        for (int j = 1;j < sum+1;j++){
            int take = arr[i-1]<=j?dp[i-1][j-arr[i-1]]:0;
            int not_take = dp[i-1][j];
            dp[i][j] = (take + not_take)%m;
        }
    }
    return dp[n][sum];
}
```

Space optimization –

```
int f(int arr[],int n,int sum){
    int cz[n+1];
    cz[0] = 1;
    int count = 0;
    for (int i = 1;i < n;i++){
        if (arr[i-1] == 0) count++;
        cz[i] = pow(2,count);
    }
    vi p(sum+1),c(sum+1);
    for (int i = 0;i< sum+1;i++) p[i] = 0;
    for (int i = 1;i < n+1;i++){
        p[0] = cz[i-1];
        for (int j = 1;j < sum+1;j++){
            int take = arr[i-1]<=j?p[j-arr[i-1]]:0;
            int not_take = p[j];
            c[j] = (take + not_take)%m;
        }
        p = c;
    }
    return p[sum];
}
```

4.5 0/1 knapsack standard

Question – You are given weights and values of N items, put these items in a knapsack of capacity W to get the maximum total value in the knapsack. Note that we have only one quantity of each item.
Find out the maximum value subset of val[] such that sum of the weights of this subset is smaller than or equal to W.

Code –

Recursive –

```
int f(int n,int w,int wt[],int val[]){
    if (!n or !w) return 0;
    int take = wt[n-1]<=w?val[n-1]+
    f(n-1,w-wt[n-1],wt,val):0;
    int not_take = f(n-1,w,wt,val);
    return max(take,not_take);
}
```

Memoization –

```
int f(int n,int w,int wt[],int val[],vvi &dp){
    if (!n or !w) return 0;
    if (dp[n][w] != -1) return dp[n][w];
    int take = wt[n-1]<=w?val[n-1]+f(n-1,w-wt[n-1],wt,val,dp):0;
    int not_take = f(n-1,w,wt,val,dp);
    return dp[n][w] = max(take,not_take);
}
```

Tabulation –

```
int f(int n,int w,int wt[],int val[]){
    vvi dp(n+1,vi(w+1));
    for (int i = 0; i < n+1;i++) dp[i][0] = 0;
    for (int j = 0; j < w+1;j++) dp[0][j] = 0;

    for (int i=1;i<n+1;i++){
        for (int j=1;j<w+1;j++){
            int take =
            wt[i-1]<=j?val[i-1]+dp[i-1][j-wt[i-1]]:0;
            int not_take = dp[i-1][j];
            dp[i][j] = max(take,not_take);
        }
    }
    return dp[n][w];
}
```

Space optimization –

```
int f(int n,int w,int wt[],int val[]){
    vi p(w+1),c(w+1);
    for (int j = 0; j < w+1;j++) p[j] = 0;
```

```
    for (int i=1;i<n+1;i++){
        for (int j=1;j<w+1;j++){
            int take =
            wt[i-1]<=j?val[i-1]+p[j-wt[i-1]]:0;
            int not_take = p[j];
            c[j] = max(take,not_take);
        }
        p = c;
    }
    return p[w];
}
```

4.6 Minimum coins

4.7 Target sum

4.8 Coin change II

4.9 Unbounded knapsack

Question – Given a set of N items, each with a weight and a value, represented by the array w[] and val[] respectively. Also, a knapsack with weight limit W.
The task is to fill the knapsack in such a way that we can get the maximum profit. Return the maximum profit.
Note: Each item can be taken any number of times.
Code –

Recursive –

```
int f(int n,int w,int val[],int wt[]){
    if (!n || !w) return 0;
    int take = wt[n-1]<=w?val[n-1]+f(n,w-wt[n-1],val,wt):0;
    int not_take = f(n-1,w,val,wt);
    return max(take,not_take);
}
```

Memoization –

```
int f(int n,int w,int val[],int wt[],vvi &dp){
    if (!n || !w) return 0;
    if (dp[n][w] != -1) return dp[n][w];
    int take = wt[n-1]<=w?val[n-1]+f(n,w-wt[n-1],val,wt,dp):0;
    int not_take = f(n-1,w,val,wt,dp);
    return dp[n][w] = max(take,not_take);
}
int f_solve(int n,int w,int val[],int wt[]){
    vvi dp(n+1,vi(w+1,-1));
    return f(n,w,val,wt,dp);
}
```

Tabulation –

```
int f(int n,int w,int val[],int wt[]){
    vvi dp(n+1,vi(w+1,0));
    for (int i = 1;i < n+1;i++){
        for (int j = 1;j < w+1;j++){
            int take = wt[i-1]<=j?
            val[i-1]+dp[i][j-wt[i-1]]:0;
            int not_take = dp[i-1][j];
            dp[i][j] = max(take,not_take);
        }
    }
    return dp[n][w];
}
```

Space optimization –

```
int f(int n,int w,int val[],int wt[]){
    vi c(w+1),p(w+1,0);
    for (int i = 1;i < n+1;i++){
```

```
        for (int j = 1;j < w+1;j++){
            int take = wt[i-1]<=j?
            val[i-1]+c[j-wt[i-1]]:0;
            int not_take = p[j];
            c[j] = max(take,not_take);
        }
        p = c;
    }
    return p[w];
}
```

4.10 Rod cutting

5 DP on strings

5.1 LCS

Question – Given two sequences, find the length of longest subsequence present in both of them. Both the strings are of uppercase.

Code –

Recursive –

```
int f(int x,int y,string s1,string s2){
    if (!x or !y) return 0;
    if (s1[x-1] == s2[y-1]) return 1+f(x-1,y-1,s1,s2);
    return max(f(x-1,y,s1,s2),f(x,y-1,s1,s2));
}
```

Memoization –

```
int f(int x,int y,string s1,string s2,vvi &dp){
    if (!x or !y) return 0;
    if (dp[x][y] != -1) return dp[x][y];
    if (s1[x-1] == s2[y-1])
        return dp[x][y] = 1+f(x-1,y-1,s1,s2,dp);
    return dp[x][y] = max(f(x-1,y,s1,s2,dp),f(x,y-1,s1,s2,dp));
}
```

Tabulation –

```
int f(int x,int y,string s1,string s2){
    vvi dp(x+1,vi(y+1,0));
    for (int i=1;i<x+1;i++){
        for (int j=1;j<y+1;j++){
            if (s1[i-1] == s2[j-1])
                dp[i][j] = 1 + dp[i-1][j-1];
            else
                dp[i][j] = max(dp[i-1][j],dp[i][j-1]);
        }
    }
    return dp[x][y];
}
```

Space optimization –

```
int f(int x,int y,string s1,string s2){
    vi p(y+1,0),c(y+1);
    for (int i=1;i<x+1;i++){
        c[0] = p[0] = 0;
        for (int j=1;j<y+1;j++){
            if (s1[i-1] == s2[j-1])
                c[j] = 1 + p[j-1];
            else
                c[j] = max(p[j],c[j-1]);
```

```
        }
        p = c;
    }
    return p[y];
}
```

5.2 Print LCS

Question – Print any of the LCS in given two strings.

Code –

```
vvi get_lcs_dp(int n,int m,vi &a,vi &b){
    vvi dp(n+1,vi(m+1,0));
    for (int i = 1;i < n+1;i++){
        for (int j = 1;j < m+1;j++){
            if (a[i-1]==b[j-1])
                dp[i][j] = 1 + dp[i-1][j-1];
            else
                dp[i][j] = max(dp[i-1][j],dp[i][j-1]);
        }
    }
    return dp;
}
vi f_solver(vi &a,vi &b){
    int n = a.size();
    int m = b.size();
    vvi dp = get_lcs_dp(n,m,a,b);
    vi ans;
    while (n && m){
        if (a[n-1] == b[m-1]){
            ans.push_back(a[n-1]);
            --n;
            --m;
        }
        else if (dp[n-1][m] > dp[n][m-1]) n--;
        else m--;
    }
    reverse(ans.begin(),ans.end());
    return ans;
}
```

5.3 Longest Common Substring

6 DP on stocks

6.1 Best time to buy and sell stocks

6.2 Buy and sell stocks II

6.3 Buy sell stocks III

6.4 Buy and stocks IV

6.5 Buy sell stocks with cooldown

6.6 Buy and sell stocks with transaction fell